ДЛЯ КАНДИДАТА / ТЕСТОВОЕ ЗАДАНИЕ

Mobile Engineer: импорт банковской выписки без потери состояния

Соберите небольшой, но надёжный фрагмент мобильного приложения: выбрать синтетическую выписку, отправить её байты локальному mock API, дождаться асинхронного результата, пережить потерю ответа и перезапуск, затем безопасно удалить импорт. Здесь важнее поведение при сбоях, чем объём экранов.

Рамки: ориентир до 8 часов работы в течение 4 календарных дней. Базовый сценарий важнее полировки. Если не успели, сдайте рабочую часть, назовите непроверенные платформы и оставшиеся риски.

Что вы получаете

- Вместе с этим PDF выдаётся **mobile-engineer-candidate-kit.zip**: локальный HTTP mock, одинаковые синтетические файлы и автотесты самого mock. Запуск: Node.js 20+, **npm start**; внешние доступы и ключи не нужны.
- Рабочий стек - React Native + TypeScript; Expo development build допустим. Проверяется нативный запуск, не web и не Expo Go. Сервер и фикстуры не изменяйте; добавлять код нужно только в мобильный клиент и его тесты.
- Реальный парсер PDF, AI, банковские интеграции, авторизация, оплата, публикация тестового приложения и свои облачные сервисы не требуются. Данные полностью вымышленные.

Сценарий

- Выбор PDF/CSV через системный picker, экран подтверждения (имя, размер, формат), отклонение ложного PDF и файла больше 10 МиБ **до POST**. Проверка не должна опираться только на расширение или заявленный MIME.
- По подтверждению приложение отправляет реальные байты локальному HTTP mock. Создание возвращает job, а готовый отчёт появляется позже. Покажите queued/processing/ready/error и возможность восстановить неопределённый исход POST.
- Результат mock - два вымышленных регулярных списания с суммой в AUD и датой. Кнопка «Подробнее» открывает карточку; никакой отмены подписки здесь нет.
- Удаление - отдельная асинхронная операция. Пока mock не вернул status=deleted, нельзя говорить, что серверные данные удалены. Свои временные копии файла удаляйте при завершении/отмене сценария.

Контракт локального HTTP mock

Клиент подключается к серверу из набора; запросы идут по сети, не через подмену вызова в UI. POST /imports принимает raw bytes (**application/octet-stream**), заголовки **Idempotency-Key** и **X-File-Name**. Ключ создаётся и сохраняется **до первой отправки**. Адрес localhost на эмуляторе настраивается по README набора.

HTTP	Вход	Выход / правило
POST /imports	Байты + стабильный ключ	202: jobId, queued, revision=1. Повтор тех же байтов и ключа - тот же job. Иные байты с тем же ключом - 409.
GET /imports/by-key/{key}	Ключ после потерянного ответа	Возвращает созданный job; 404 означает «пока не найден». При гонке повторяйте POST с тем же ключом.

HTTP	Вход	Выход / правило
GET /imports/{jobId}	jobId	queued / processing / ready / deleting / deleted + revision. Готовый результат только при ready.
DELETE /imports/{jobId}	Стабильный ключ удаления	202: deleting. Удаление подтверждено только при последующем GET: deleted.

Фиксированный ready-ответ

status="ready"; revision=3; currency="AUD"

rec-001: StreamBeam, amountMinor=1999, monthly, lastSeen=2026-10-01

rec-002: BrightGrid, amountMinor=13200, monthly, lastSeen=2026-10-03

Это ответ сервера, а не результат клиентского парсинга. Никаких реальных банковских файлов и персональных данных. Не выводите исходные байты, имя файла или описания операций в аналитику и crash-логи.

Одинаковые файлы для проверки

Файл из набора	Ожидание
statement.csv, statement.pdf	Оба принимаются; после ready видны две позиции и сумма 151.99 AUD/мес.
spoofed.pdf	Текст внутри файла, не PDF. Клиент отклоняет до POST.
oversize.csv	Создаётся командой npm run fixtures:oversize. Клиент отклоняет >10 МиБ до POST.

Единая матрица приёмки

Сценарий набора	Наблюдаемое условие приёмки
happy	Файл реально отправлен; 202 не объявлен готовым отчётом. После ready видны данные mock.
lost_ack	Первый POST создал job без ответа. После retry/relaunch GET по ключу находит его; GET /__test/state показывает newJobs=1.
stale_status	После ready/revision 3 приходит processing/revision 2. UI не откатывается; тест это фиксирует.
delete_failure	DELETE вернул 503. UI не утверждает удаление; после переключения в happy повтор с тем же ключом доходит до deleted.
spoofed + oversize	Оба файла отвергнуты до POST; сравните createRequests в /__test/state до и после.

Инженерные требования

- Продумайте временный доступ к Android content:// и iOS Files URL, чтение байтов, ограничение размера без загрузки всего большого файла в память и очистку своей приватной временной копии. Не доверяйте расширению/MIME без проверки содержимого.
- Сохраните ключ и минимальное состояние операции до POST. После обычного relaunch покажите актуальный job без самопроизвольной повторной отправки файла. Поздний ответ не должен воскресить удалённый job.

- Сценарии mock переключаются по /__test/scenario; допустим простой экран диагностики или команды из README. Не меняйте исходники между прогонами. В логах и аналитике нет исходных байтов, имени файла и описаний операций.
- Добавьте тесты на сохранённый ключ, один job при lost_ack, устаревшую ревизию, неуспешное удаление и локальное отклонение двух плохих файлов. Результат тестов и счётчики mock должны быть воспроизводимы.
- Покажите нативный запуск на iOS и Android: хотя бы выбор файла и один полный импорт на каждой платформе. Если доступна только одна, честно отметьте вторую как непроверенную; её баллы нельзя заменить словами или веб-сборкой.
- Небольшой экран и крупный системный шрифт не ломают основные действия. Читабельный loading/error/empty state важнее декора.

Релизная зрелость - без доступа к сторам

- В README: точный clean checkout, команда запуска клиента и mock, тесты, устройства/версии ОС и фактический результат отдельно для iOS и Android. Не отмечайте непроверенную платформу как работающую.
- Короткий release note: какие разрешения действительно нужны этому фрагменту, где остаются копии файла, как очистить данные, какой сигнал остановит rollout, что потребует нового нативного бинарника вместо JS/OTA.
- Подтверждение вашего реального опубликованного приложения проверяется на техническом интервью до этого задания; в репозиторий тестового не добавляйте доступы к сторам и чужие рабочие материалы.

Что прислать

- Репозиторий клиента и неизменённого набора, README и результаты тестов. Короткое видео: iOS, Android, happy, lost_ack/relaunch, stale_status, delete_failure; если часть не проверена, покажите честно.
- Self-review до одной страницы: 2-3 принятых решения, что упростили ради срока, что сломается первым в production и как это заметите. AI-инструменты разрешены; опишите, что и как проверили самостоятельно.
- Публичный URL, реальные банковские файлы, внешние API, пароли и доступ в магазины не нужны. Не публикуйте тестовое приложение в сторы.

Как оцениваем

Блок	Вес	Проверяемое доказательство
Передача и состояние	30	Байты через HTTP, один job, recovery, ревизии, удаление
Обе нативные платформы	20	Picker, жизненный цикл файла, запуск/проверка каждой ОС
Приватность	20	Локальная валидация, копии, очистка, отсутствие утечек
Тесты и диагностика	15	Матрица с воспроизводимыми тестами и счётчиками mock
Интерфейс	10	Правдивые состояния, ошибки, крупный шрифт, маленький экран
Релизная записка	5	Факты об ОС, рисках и границе OTA/бинарника

Шкала: каждый блок 0-4: 0 - нет/опасно; 1 - только демо; 2 - частично, без проверок; 3 - все базовые сценарии воспроизводятся; 4 - плюс отказные случаи, тесты и объяснённые trade-offs. Баллы блока = вес x уровень / 4. Критическая утечка документа/секрета, второй job после повторной отправки или ложное подтверждение удаления - стоп-фактор независимо от красивого UI.

Как проверим сдачу

Проверяющий самостоятельно запустит проект из clean checkout, пройдёт сценарии из матрицы и сверит счётчики локального mock с состоянием интерфейса и автотестами. Дополнительного кодинга, изменений после дедлайна или доступа к внешним сервисам для приёмки не требуется.